JSON Web Tokens (JWTs)

Part I

JSON Web Token (JWT)

A URL-safe and cryptographically secured object representing claims to be transferred between two parties. Part of the JOSE Framework.

JSON Object Signing and Encryption (JOSE)

Core JOSE RFCs (there are MANY!):

RFC	Name (Short)	Purpose
RFC 7515	**JSON Web Signature (JWS)**	Represents content secured with digital signatures or MACs
RFC 7516	**JSON Web Encryption (JWE)**	Represents encrypted content using JSON structures
RFC 7517	**JSON Web Key (JWK)**	JSON data structure representing a cryptographic key
RFC 7518	**JSON Web Algorithms (JWA)**	Defines signing, encryption, and key mgmt algorithms
RFC 7519	**JSON Web Token (JWT)**	Compact claims representation using JWS or JWE
RFC 7520	**Examples of JOSE**	Examples of using JOSE to protect content
RFC 8725*	**JWT Best Current Practices**	Security guidance updating RFC 7519 (BCP 225)

* New updates coming: https://datatracker.ietf.org/doc/draft-ietf-oauth-rfc8725bis/

JSON Web Signature (JWS)

- Defines use of digital signatures or Message Authentication Codes (MACs) to secure (*Integrity* + *Authenticity*) JSON-based data
- Tokens are **NOT** encrypted; all contents can be viewed
- Uses JSON data structures with Base 64 (URL) encoding
- Consists of:
  - Header (*JOSE Header*)
  - Payload
  - Signature
- Supports various non-JWT formats
- Unlike JWTs, payloads can be any non-JSON data

JSON Web Signature (JWS)

Token Structure (Compact Serialization – URL Safe):

BASE64URL(header).BASE64URL(payload).BASE64URL(signature)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3
ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiYWRtaW4iOnRydWUsImlh
dCI6MTUxNjIzOTAyMn0.KMUFsIDTnFmyG3nMiGM6H9FNFUROf
3wh7SmqJp-QV30

Signature Computation (Protected Header)

alg(ASCII(base64url(UTF-8(header)) + "." + base64url(payload), <*secret*>))

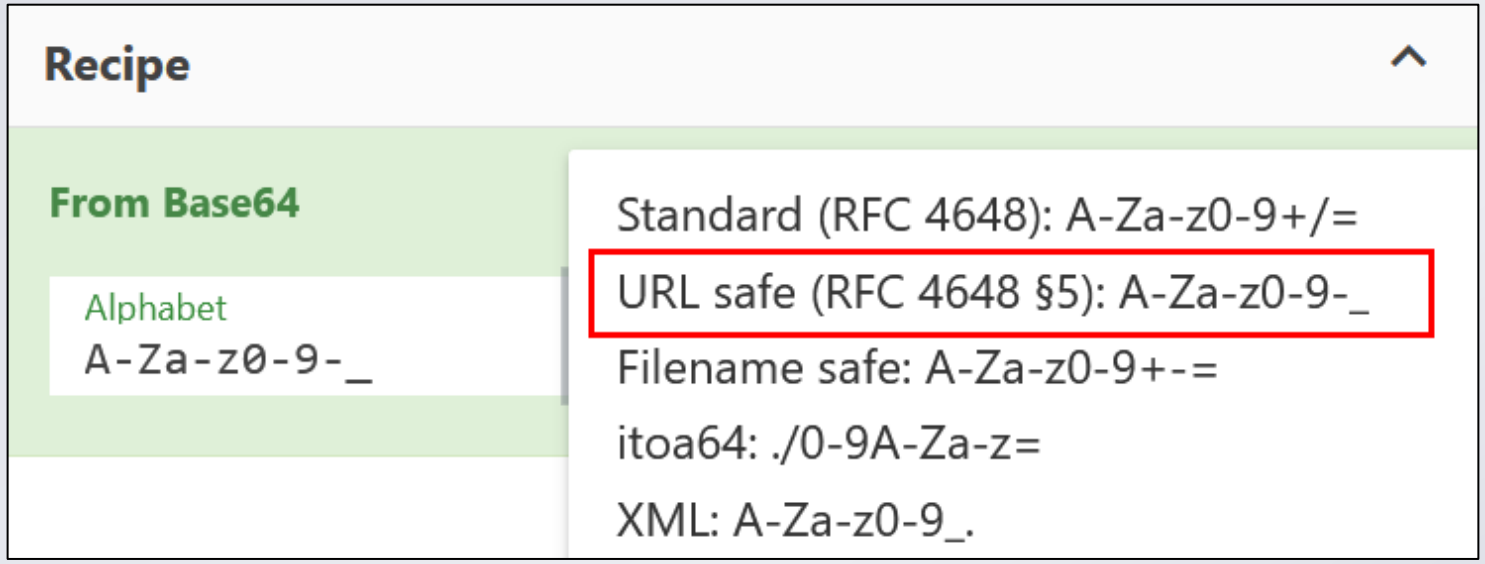
JSON Web Signature (JWS)

NOTE: Base64url from RFC 7515:

Base64 encoding using the URL- and filename-safe character set defined in Section 5 of RFC 4648 [RFC4648], with all trailing '=' characters omitted (as permitted by Section 3.2) and without the inclusion of any line breaks, whitespace, or other additional characters. Note that the base64url encoding of the empty octet sequence is the empty string.

JSON Web Signature (JWS)

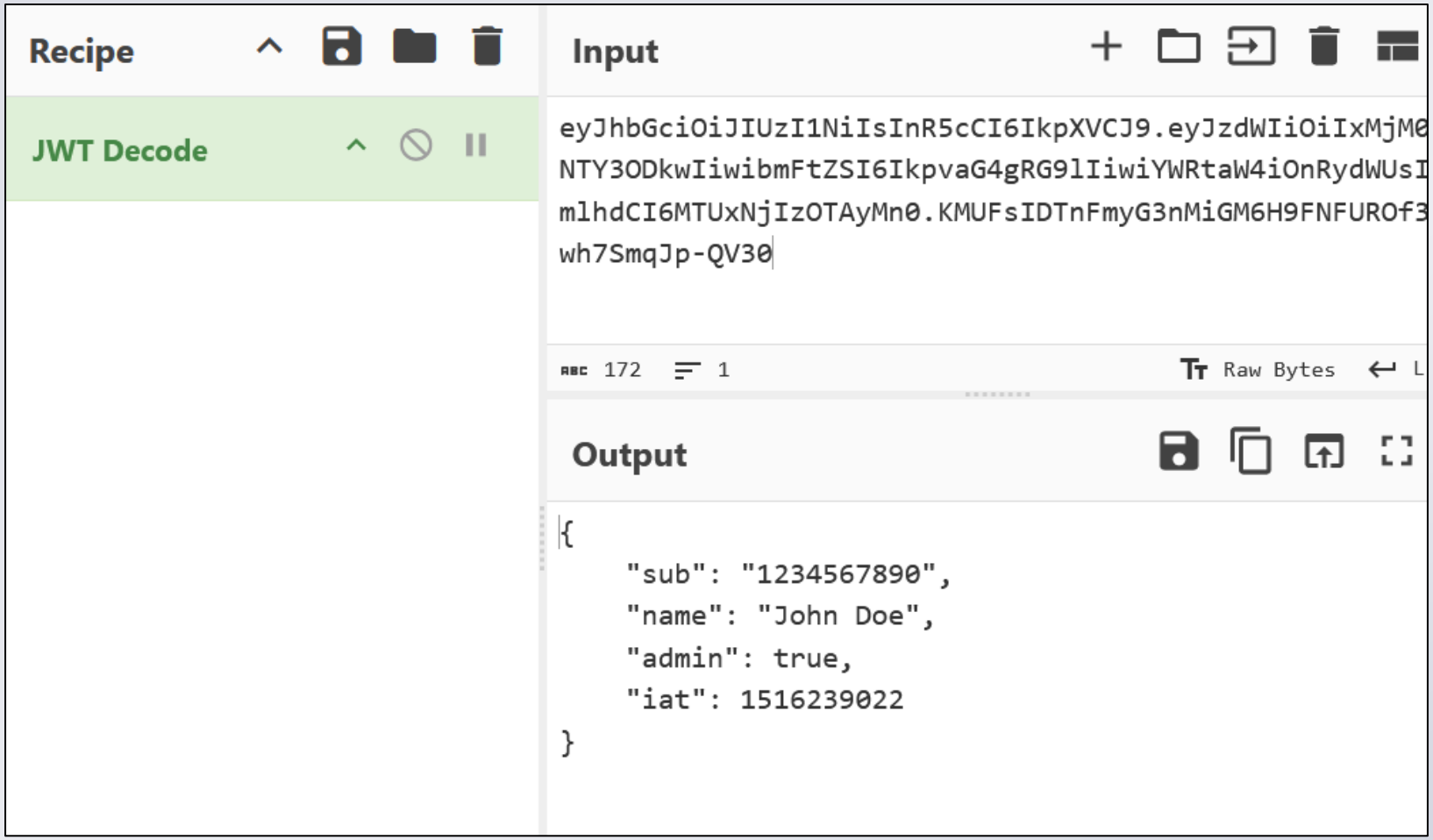
CyberChef From Base64 Recipe:



NOTE: because of padding stripping ('=') Base64 parsers may fail (avoid 'strict' modes if just looking for a simple decode)

JSON Web Signature (JWS)

JSON Web Signature (JWS)



Aside: Identifying Base64 JSON

Base64({“)

↓

ey...

(often eyJ..)

JSON Web Signature (JWS)

Token Structure (JSON Serialization Form):


```
{
  "payload": "eyJpc3MiOiJqb2UiLCJleHAiOjEzMDA4MTkzODA...",
  "signatures": [
    {
      "protected": "eyJhbGciOiJSUzI1NiJ9",
      "header": {
        "kid": "2010-12-29"
      },
      "signature": "cC4hiUPoj9Eetdgtv3hF80EGrhuB..."
    },
    {
      "protected": "eyJhbGciOiJFUzI1NiJ9",
      "header": {
        "kid": "e9bc097a-ce51-4036-9f76-9c6d4a0ce5d7"
      },
      "signature": "DtEhU3ljbEg8L38VWAfUAqOyKAM6-Xx-F4..."
    }
  ]
}
```

JOSE Header

- **Describes the algorithm applied and optional properties (parameters)**
- **Parameter names must be unique**
- **The *alg* parameter is mandatory for JWS/JWE**
- **Structured as JSON serialized as URL-safe Base 64**
- **A "Protected Header" is covered by the signature**
- **Examples:**

```
{  "alg": "HS256"}

{  "alg": "RS256",  "typ": "JWT",  "kid": "key-12345"}

{  "alg": "HS256",  "cty": "JWT"}
```

JOSE Header: Registered Parameters (JWS)

Defined In	Applies To	Parameter	Purpose
RFC 7515,7516	JWS, JWE	**alg**	**Required**: indicates the signing/encryption algorithm
RFC 7515	JWS, JWE	**jku**	URL indicating JWK Set containing keys
RFC 7515	JWS, JWE	**jwk**	Embedded JSON Web Key
RFC 7515	JWS, JWE	**kid**	Key identifier
RFC 7515	JWS, JWE	**x5u**	URL to X.509 Certificate
RFC 7515	JWS, JWE	**x5t**	X.509 certificate chain
RFC 7515	JWS, JWE	**x5t#S256**	SHA-256 thumbprint of cert
RFC 7515	JWS, JWE	**typ**	Type of object (e.g., JWT)
RFC 7515	JWS, JWE	**cty**	Content type of the payload
RFC 7515	JWS, JWE	**crit**	List of critical headers that must be understood

JOSE Header: SAVE THIS LINK!

JSON Object Signing and Encryption (JOSE) Reference:

https://www.iana.org/assignments/jose/jose.xhtml

JSON Web Signature and Encryption Header Parameters

Registration Procedure(s)
Specification Required
Expert(s)
Sean Turner, Mike Jones, Filip Skokan
Reference
[RFC7515]
Note
Registration requests should be sent to the mailing list described in [RFC7515]. If approved, designated experts should notify IANA within three weeks. For assistance, please contact iana@iana.org.

Available Formats
CSV

Header Parameter Name	Header Parameter Description	Header Parameter Usage Location(s)	Change Controller	Reference
alg	Algorithm	JWS	[IESG]	[RFC7515, Section 4.1.1]
jku	JWK Set URL	JWS	[IESG]	[RFC7515, Section 4.1.2]
jwk	JSON Web Key	JWS	[IESG]	[RFC7515, Section 4.1.3]
kid	Key ID	JWS	[IESG]	[RFC7515, Section 4.1.4]
x5u	X.509 URL	JWS	[IESG]	[RFC7515, Section 4.1.5]

JSON Web Encryption (JWE)

- Unlike JWS, provides *Confidentiality* by encrypting payloads
- Consists of 5 parts in compact serialization:
  - Header
  - Content Encryption Key (CEK)
  - IV
  - Ciphertext
  - Authentication Tag (integrity for ciphertext)
- Supports multiple algs and key types (symmetric/asymmetric)
- Less common than JWS variants, but still popular
- Compared to JWS, adds registered parameters *enc* and *zip*
  - *enc* specifies encryption algorithm and *zip* compresses before encrypting

JSON Web Encryption (JWE) Examples

Consider the JWE Header:

```
{ "alg": "RSA-OAEP", "enc": "A256GCM", "typ": "JWE", "kid": "rsa-key-1"}
```

- **alg – The algorithm is RSA-OAEP, which is used to encrypt the CEK**
  - **Aside: the CEK is the symmetric key generated per-message to encrypt the payload and is embedded in the token encrypoted**
    - **Why? Primarily, symmetric encryption is more efficient for full payload encryption**
- **enc – The algorithm used by the CEK to encrypt the payload**
- **typ – The type of token (JWE)**
- **kid – This indicates the key used to encrypt the CEK via the *alg***

JSON Web Encryption (JWE) Examples

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

☑ Remove non-alphabet chars

☐ Strict mode

Input


```
eyJlbmMiOiJBMjU2R0NNIiwiYWxnIjoiUlNBLU9BRVAtMjU2In0.O01BFr_XxGzKEUb_Z9vQOW3DX2cQFxojrRy2JyM5_nqKnrpAa0rvcPI_ViT2PdP
p1cgzAWBQzm1QYL8fwaz16MS48ExRz41dLhA0aEWE4e7TYzjrfaK8M4wIUlQCFIl-
wS1N3U8W2XeUc9MLYGmHft_Rd9KJs1c-9KKdUQf6tEzJ92TGEC7TRZX4hGdtszIq3GGGBQaW8P9jPozqaDdrikF18D0btRHNf3_57sR_CPEGYX0O4mY
fZ0xoA.ln7IYZDF9TdBIK6i.ZhQ3Q5TY827KFQw8DdRRzQVJVFdIE03B6AxMNZ1sQIjlUB4QUxg-UYqjPJESPUmFsODeshGWLa5t4tUri5j6uC4mFDb
```


543 1

Output


```
{"enc":"A256GCM","alg":"RSA-OAEP-256"}ÓPE•|FÌ¡m•o@å·}•@\h•´rØ••æz••º@kJïpò•=•tôh•#dC3k•-ÝAmêÊBV•bê•yéÌ4#¢Át
ÀX3•T/Çõk=z1.<sãWK•a8{´ØÎ:ßh¯
ã•"\ÔÝÔñm•yG=0¶•wíEßJ&Í\ô¢•Q´ú´LÉ÷dÆ.ÓE•ø•gm³2*Üa•ðÿc>•êh7k•Auð=µÍ•~{±•f•ÐÎ&c¾ù•µª•5iÓg}•1 gì•
_St•ê&aCt9M•6ì¡PÃÀÝE•ÐT•Et•4Ü••ÄÃÝÖÄ••Uá1•F*•òDHõ&•Ã•zÈFX¶¹·•T®.cêà¸•PÛ••¦>cJ!••æo2s••• 1••»ÉÕQ@¢ZOH
```


But why are *typ* and *kid* omitted? ONLY *enc* and *alg* are required.

JSON Web Encryption (JWE) Examples

Component	Encoded (Base64url)	Decoded/Value
Protected Header	eyJhbGciOiJSU0EtT0FFUCIsImVuYyI6IkEyNTZHQ00ifQ	{"alg":"RSA-OAEP","enc":"A256GCM"}
Encrypted Key (CEK)	OKOawDo13gRp2ojaHV7LFpZcgV7T6DVZKTyKOMTYUmKoTCVJRg3R3dh0v1	38 E2 6B 00 3A 35 DD 06 69 D2 8A 36 87 57 BC B1 69 65 C8 15 BB 4F A0 D5 93 24 F2 28 E3 13 61 49 8A A1 30 95 25 18 37 47 77 61 D1 BF (**encrypted**)
Initialization Vector (IV)	48V1_ALb6US04U3b	E1 75 7F 00 B5 BE 94 4B 4D 38 53 7B
Ciphertext	5eym8TW_c8SuK0ltJ3rpYIzOeDQz7TALvtu6UG9oMo	E7 B2 9B C4 D6 FD F3 12 B8 AD 25 B7 A7 A5 82 33 39 E0 D0 CF B4 C0 2E FB 6E E9 41 BD A0 CA (**encrypted**)
Authentication Tag	XFBoMYUZodetZdvTiFvSkQ	5C 50 68 31 84 59 A1 D7 AD 65 DB D3 88 5B D2 91

JWS vs. JWE

JSON Web Signature (JWS)

3 parts separated by dots:

```
header.payload.signature
```

- Provides integrity protection via digital signature or MAC
- Payload is Base64URL-encoded (readable, NOT encrypted)
- Used for: authentication tokens, ID tokens, signed assertions

JSON Web Encryption (JWE)

5 parts separated by dots:

```
header.key.iv.ciphertext.tag
```

- Provides confidentiality — payload is encrypted, not just encoded
- Uses key management algorithm + content encryption algorithm
- Used for: sensitive claims, encrypted assertions, nested JWTs

JSON Web Tokens (JWT)

- A *profile* (type) of JWS or JWE with a strictly defined structure
- Payload MUST be valid JSON (RFC 7519) with *Compact Serialization*
- The payload is called the *Claims Set*
- A JWT can be JWS/JWE; not all JWS/JWE are JWT!
- Similar to JOSE Header parameters, Registered Claims are defined by the RFC
- Claims optional and are classified as:
  - Registered – these are defined in the RFC
  - Public – these are typically intended for shared use and should be in the IANA JSON Web Token Claims registry
  - Private – Unregistered, typically application-specific claims

JSON Web Tokens (JWT): Common Claims

Claim	Name	Purpose
iss	Issuer	Indicate who issued the token
sub	Subject	Indicate the subject of the token
aud	Audience	Indicate the intended recipient of the token
exp	Expiration Time	Indicates an expiration time (Unix timestamp / Epoch time)
nbf	Not Before	Enforced a not valid before time (Unix timestamp / Epoch time)
iat	Issued At	Issued at time (Unix timestamp / Epoch time)
jti	JWT ID	An identifier for the JWT

JWT Claims: SAVE THIS LINK!

JSON Web Token (JWT) Reference:

https://www.iana.org/assignments/jwt/jwt.xhtml

JSON Web Token Claims

Registration Procedure(s)
Specification Required
Expert(s)
Brian Campbell, Mike Jones, Nat Sakimura, Filip Skokan
Reference
[RFC7519]
Note
Registration requests should be sent to the mailing list described in [RFC7519]. If approved, designated experts should notify IANA within three weeks. For assistance, please contact iana@iana.org.

Available Formats
CSV

Claim Name	Claim Description	Change Controller	Reference
iss	Issuer	[IESG]	[RFC7519, Section 4.1.1]
sub	Subject	[IESG]	[RFC7519, Section 4.1.2]
aud	Audience	[IESG]	[RFC7519, Section 4.1.3]
exp	Expiration Time	[IESG]	[RFC7519, Section 4.1.4]
nbf	Not Before	[IESG]	[RFC7519, Section 4.1.5]
iat	Issued At	[IESG]	[RFC7519, Section 4.1.6]
jti	JWT ID	[IESG]	[RFC7519, Section 4.1.7]
name	Full name	[OpenID_Foundation_Artifact_Binding_Working_Group]	[OpenID Connect Core 1.0, Section 5.1]
given_name	Given name(s) or first name(s)	[OpenID_Foundation_Artifact_Binding_Working_Group]	[OpenID Connect Core 1.0, Section 5.1]

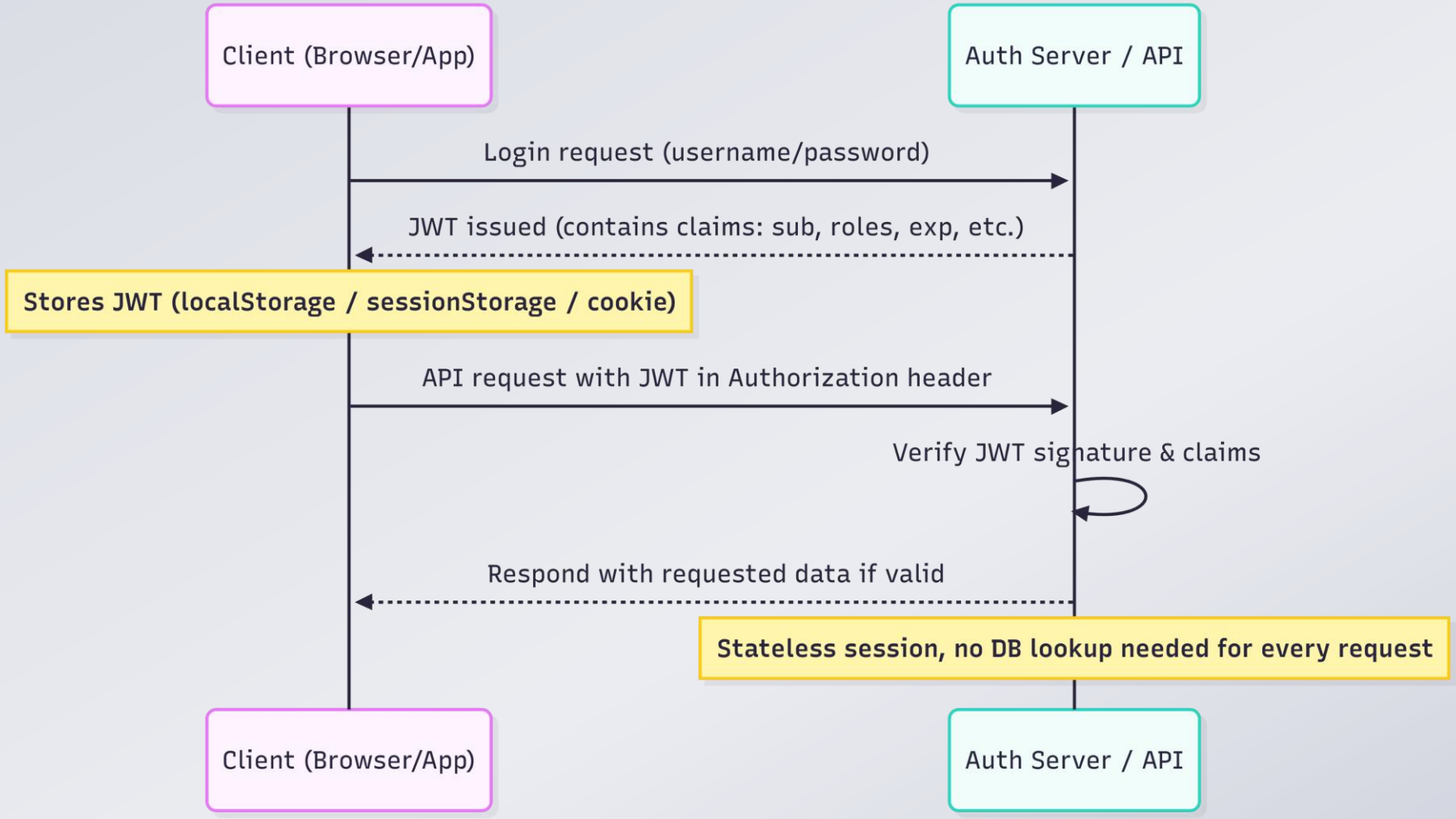
JSON Web Tokens (JWT): Common Usage

- Most used as a *self-contained* token. From OWASP ASVS V5:

  Self-Contained Token – A token that encapsulates one or more attributes that do not rely on server-side state or other external storage. These tokens ensure the authenticity and integrity of their contained attributes, enabling secure, "stateless" information

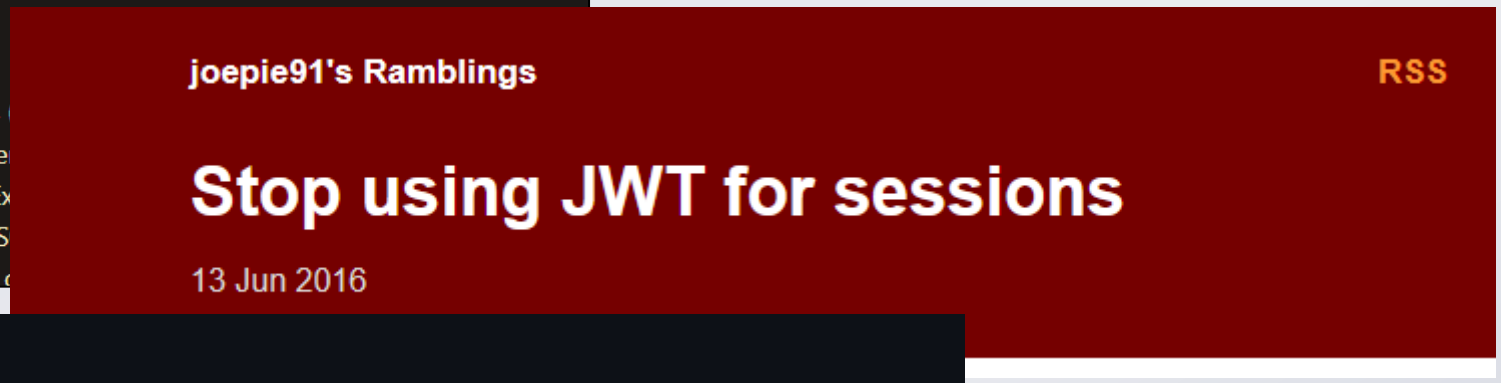
- JWTs primarily enable secure transmission of *claims* between systems (an issuer and audience), but…
- Commonly, JWTs are used in place of a “stateful” session management mechanism (more on this soon…)
- Used as part of OpenID Connect and OAuth 2.0

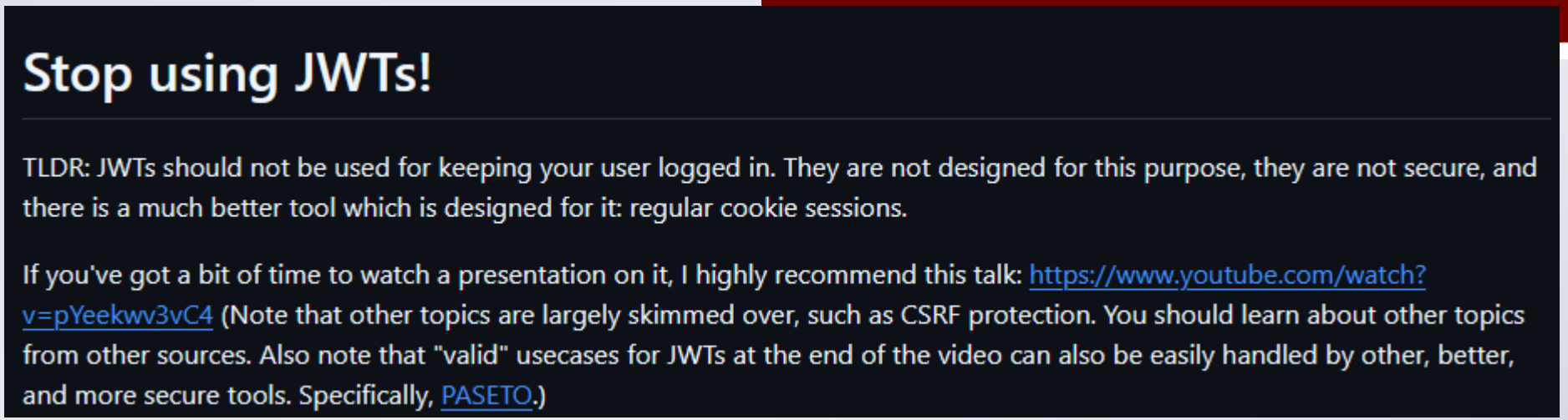
JSON Web Tokens (JWT): Common Usage



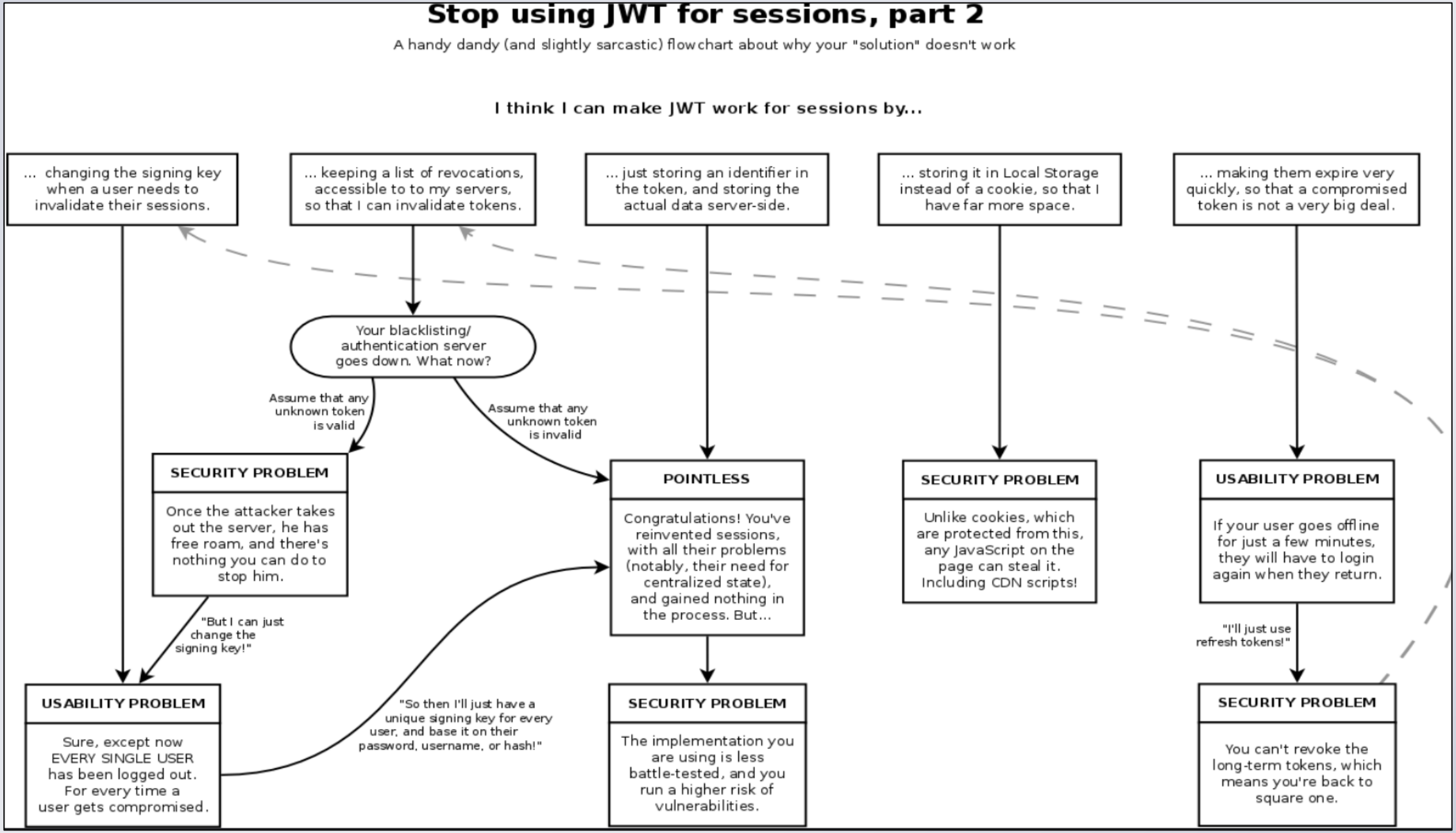
JWT Session Tokens

In naïve/simple implementations, these cannot be revoked!



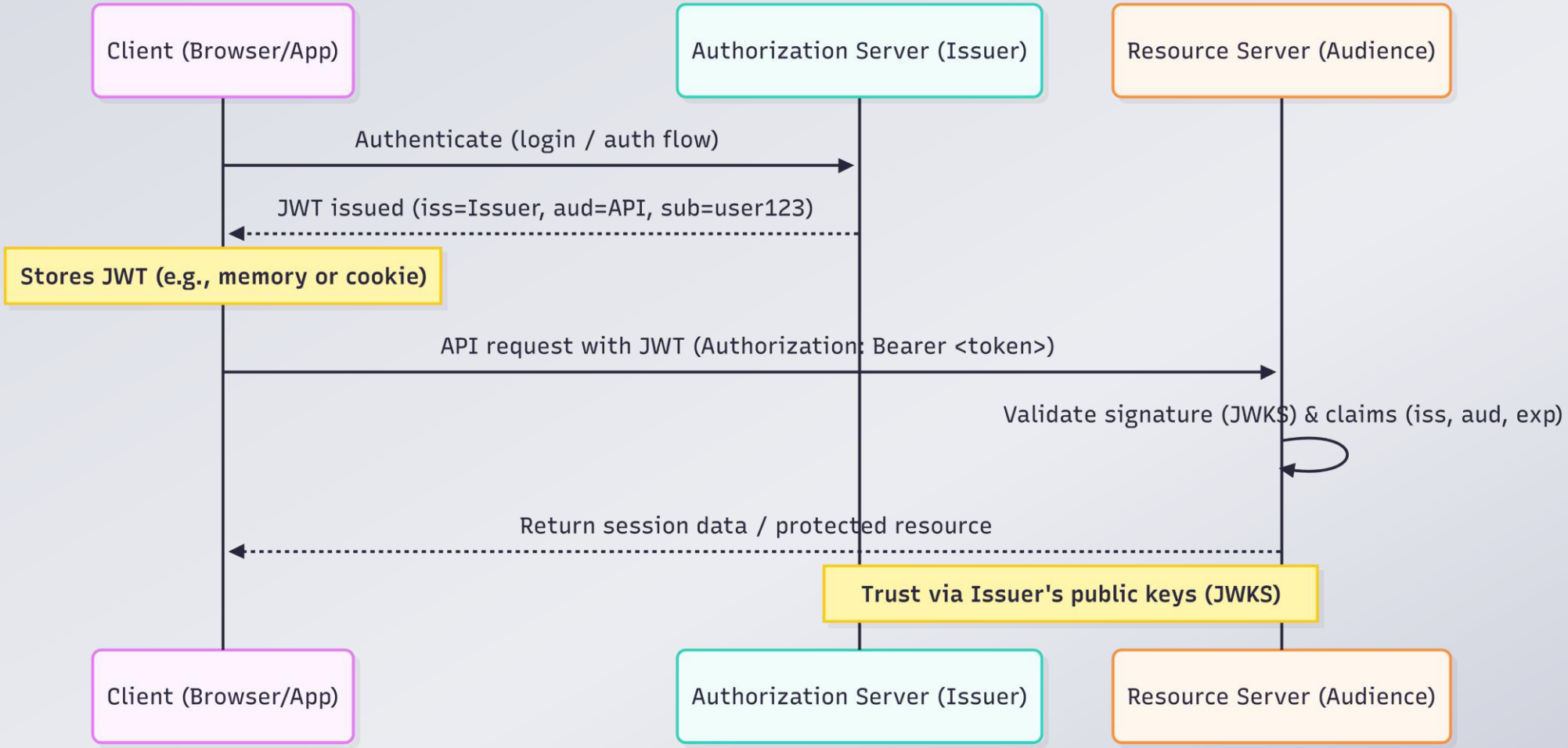


https://ianlondon.github.io/posts/dont-use-jwts-for-sessions/
http://cryto.net/~joepie91/blog/2016/06/13/stop-using-jwt-for-sessions/
https://gist.github.com/samsch/0d1f3d3b4745d778f78b230cf6061452



The Parts of JWT Security Nobody Talks About | Philippe De Ryck, Google Developer Expert
https://www.youtube.com/watch?v=DPrhem174Ws

JSON Web Tokens (JWT): Oauth 2.0 / OIDC



JSON Web Key (JWK)

- Standard to represent keys in JSON format (RFC 7517)
  - Useful to transmit keys on the web (either symmetric or asymmetric)
- Consist or *required* and *optional* JWK members
- Simple example: RSA public key

```
{
  "kty": "RSA",
  "use": "sig",
  "kid": "rsa1",
  "alg": "RS256",
  "n": "sXchvExrYKvXsG4WQdG7N7O87C_4v0F5WzBqg3xTtX8e3Z8s1eA7tXQz1D8RlkxQy0d8I8wXvXw6c1tR2n_dqzvTg56xg1m6dD...",
  "e": "AQAB"
}
```

JSON Web Key (Key): Members

Member	Required?	Key Type	Purpose
kty	Yes	All	Specifies the key type
suuseb	No	All	Indicates public key use (*sig* or *enc*)
key_ops	No	All	Permitted operations (*sign*, *verify*...)
alg	No	All	Intended algorithm (*RS256*, *A256GCM*...)
kid	No	All	Key identifier to select appropriate key
n, e, d, p, q...	Depends	RSA keys	Components of RSA keys (note: some are private members)
crv, x, y	Depends	EC keys	Components of EC keys

JSON Web Key Sets (JWKS)

Multiple keys can be bundled into a JWK set, making it a JWKS

```
{
  "keys": [
    {
      "kty": "RSA",
      "use": "sig",
      "kid": "rsa1",
      "alg": "RS256",
      "n": "...",
      "e": "AQAB"
    },
    {
      "kty": "oct",
      "k": "GawgguFyGrWKav7AX4VKUg",
      "use": "enc",
      "kid": "sym1",
      "alg": "A256GCM"
    }
  ]
}
```

JSON Web Key (JWK) On The Web

Public Keys are often published as part of JWKS endpoints

- **OpenID Connect (OIDC) / OAuth 2.0 Providers**
  - **/.well-known/jwks.json**
- **AWS Cognito**
  - **https://cognito-idp.<region>.amazonaws.com/<userPoolId>/.well-known/jwks.json**
- **Microsoft / Azure AD / Entra ID**
  - **https://login.microsoftonline.com/<tenant-id>/discovery/v2.0/keys**
- **Possible security issue: publication of sensitive key material (for example, private members of RSA)**

JSON Web Algorithms (JWA)

- **Per RFC 7518, this specification registers algorithms with identifiers**
- **Standardizes names and usage for JWS, JWK, and JWE**
- **Once again: https://www.iana.org/assignments/jose/jose.xhtml**

Summary:

Algorithm Type	Example Names	Purpose
Signature / MAC	HS256, RS256, ES256, PS512	Symmetric (HMAC) or Asymmetric (RSA/ECDSA); used in JWS/JWT signatures
Key Managment	RSA-OAEP, A256KW, dir, ECDH-ES	Symmetric or Asymmetric; used in JWE key encryption
Content Encryption	A256GCM, A128CBC-HS256	Symmetric; used in JWE payload encryption

JSON Web Token Best Current Practices

- Current draft: https://datatracker.ietf.org/doc/draft-ietf-oauth-rfc8725bis/
- To be covered in the next session…